

Practical - 28: Implementation of exception handling mechanism

110) Demonstrate the built-in types of exception using example. (A)

```
public class p110{
    public static void main(String[] args) {

        // ArithmeticException
        try {
            int a = 10 / 0;
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception: " + e);
        }

        // ArrayIndexOutOfBoundsException
        try {
            int arr[] = {10, 20, 30};
            System.out.println(arr[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Array Index Exception: " + e);
        }

        // NullPointerException
        try {
            String str = null;
            System.out.println(str.length());
        } catch (NullPointerException e) {
            System.out.println("Null Pointer Exception: " + e);
        }

        // NumberFormatException
        try {
            int num = Integer.parseInt("ABC");
        } catch (NumberFormatException e) {
            System.out.println("Number Format Exception: " + e);
        }

    }
}
```

111) Demonstrate the customize exception using throw keyword with example. (A)

```
class InvalidAgeException extends Exception {
    InvalidAgeException(String msg) {
        super(msg);
    }
}

public class p111 {
    public static void main(String[] args) {
        int age = 15;
```

```
try {
    if (age < 18) {
        throw new InvalidAgeException("Age must be 18 or above");
    }
    System.out.println("Eligible to vote");
} catch (InvalidAgeException e) {
    System.out.println("Exception: " + e.getMessage());
}
}
```

- 112)** WAP to accept N integer numbers from the command line. Raise and handle exceptions for following cases : (B)
- when a number is -ve
 - when a number is evenly divisible by 10
 - when a number is greater than 1000 and less than 2000
 - when a number is greater than 7000 Skip the number if an exception is raised for it, otherwise add it to find total sum.

```
class NegativeNumberException extends Exception {
    NegativeNumberException(String msg) {
        super(msg);
    }
}

class DivisibleByTenException extends Exception {
    DivisibleByTenException(String msg) {
        super(msg);
    }
}

class RangeException extends Exception {
    RangeException(String msg) {
        super(msg);
    }
}

class GreaterThan7000Exception extends Exception {
    GreaterThan7000Exception(String msg) {
        super(msg);
    }
}

public class p112 {
    public static void main(String args[]) {

        int sum = 0;

        for (String s : args) {
```

```

        try {
            int num = Integer.parseInt(s);

            if (num < 0)
                throw new NegativeNumberException("Negative Number");

            if (num % 10 == 0)
                throw new DivisibleByTenException("Divisible by 10");

            if (num > 1000 && num < 2000)
                throw new RangeException("Between 1000 and 2000");

            if (num > 7000)
                throw new GreaterThan7000Exception("Greater than 7000");

            sum += num;

        } catch (Exception e) {
            System.out.println(numMessage(s) + " -> " + e.getMessage());
        }

        System.out.println("Total Sum = " + sum);
    }

    static String numMessage(String s) {
        return "Skipped Number " + s;
    }
}

```

113) WAP to create Account class, which is representing a bank account where we can deposit and withdraw money. if we want to withdraw money which exceed our bank balance? We will not be allowed, create a customize exception to handle above situation and display proper error message. (C)

```

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) {
        super(msg);
    }
}

class Account {
    double balance;

    Account(double balance) {
        this.balance = balance;
    }

    void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
    }
}

```



```
void withdraw(double amount) throws InsufficientBalanceException {
    if (amount > balance) {
        throw new InsufficientBalanceException(
            "Insufficient Balance! Available Balance = " + balance);
    }

    balance -= amount;
    System.out.println("Withdrawn: " + amount);
}

void display() {
    System.out.println("Balance = " + balance);
}
}

public class p113 {
    public static void main(String[] args) {

        Account a = new Account(5000);

        try {
            a.display();

            a.withdraw(2000);
            a.display();

            a.withdraw(4000); // Exception

        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        }
    }
}
```